Castles in the Air

Extended chapter commentary

CHAPTER 4
The glyph machine

© John Longley 2023

This version 2 December 2023

CHAPTER 4 COMMENTARY

The glyph machine

Having completed our whirlwind survey of mathematics as a whole in the first two chapters, we have now arrived at the main topic of this book: the branch of mathematics known as *mathematical logic*. Let's begin by saying a bit more about what this is.

In general, we can describe any branch of mathematics in terms of the sorts of concepts it studies. Geometry studies space and various shapes and forms within it; arithmetic studies numbers and operations on them; group theory studies the general concept of symmetries; and so on. So what is it that mathematical logic studies? Seen alongside other branches of mathematics, this branch may appear as something of an oddity — for as we mentioned at the end of Chapter 1, what mathematical logic seeks to study is *mathematics itself*. Whilst geometry is concerned with concepts such as point, line, plane, circle, and triangle, mathematical logic is concerned with concepts such as statement, theorem, proof, algorithm, computation and the like: concepts that we make use of all the time when doing other kinds of mathematics. (For this reason, the field of mathematical logic is also known as *metamathematics*.) So although mathematical logic is itself a branch of mathematics, its results can and do feed into philosophical discussions *about* mathematics — discussions of the sorts of questions we raised in Chapter 1.

Now, this idea of applying mathematics to itself might instantly arouse suspicions. Can it be done? Don't we risk getting caught up in some kind of vicious circle here? As we shall see later on, such suspicions are not entirely unjustified — we *can* fall into the trap of circularity if we ask the wrong kinds of questions, or if we expect too much from our enquiries. On the other hand, I think we'd agree that there's not necessarily a problem *in principle* with the idea of using something to study itself: for instance, scientists studying atoms use equipment made of atoms, while scientists studying the human brain make extensive use of (among other things) their own brains. So although a healthy dose of scepticism is quite in order here, it seems reasonable to start by giving a fair hearing to what mathematics has to say about itself — whilst remaining on our guard as to the possibly circular nature of the entire enterprise — and seeing where this leads us.

Our purpose in this chapter is to look in detail at one of the most fundamental 'raw ingredients' of mathematics — the idea of a *mathematical*

statement — and to see how this idea can itself be treated mathematically. The main idea is that mathematical statements can be expressed formally by logical sentences, such as the glyph statements in our story. Once we've explored the basic idea a bit, we'll look at some of the possibilities it opens up — in particular, the possibility of *decision algorithms* for various parts of mathematics. This will lead us on to another fundamental concept — that of *computability* — which will be the topic of the following chapter.

A language of thought

Let's first clarify what we mean on an intuitive level by a 'mathematical statement'. Roughly speaking, we just mean a complete, self-contained assertion with a mathematically precise content — something like 'there is a prime number greater than 100'. A mathematical statement (in the sense we mean here) doesn't need to be a *true* statement — for example, 'all prime numbers are less than 100' is also a mathematical statement — but it does need to be 'grammatically correct' and to involve only mathematically precise notions. Thus, 'there are a lot of prime numbers' isn't a mathematical statement as it stands, since 'a lot of' is an informal, fuzzy notion rather than a mathematically precise one. We also don't want to count 'n is a prime number' as a mathematical statement, because it isn't *self-contained* — there's nothing here to say what 'n' is or what role it's supposed to play.

So how about making the definition of mathematical statement itself mathematically precise? At first, that might seem a rather tall order. When mathematicians make such statements, they tend to use a curious mixture of ordinary human language and mathematical symbols: for example, they might say 'There are no positive whole numbers x, y, z, n with $n > 2$ and $x^n + y^n = z^n$'. So it might at first seem that in order to say exactly what a mathematical statement is, we'd need to give a mathematically precise description of all relevant aspects of (for instance) English grammar as well as the mathematical symbolism. And even supposing we could do this, there would be other problems: for instance, English itself isn't a perfect vehicle for mathematical expression, since even grammatically correct sentences can be ambiguous in subtle ways. What we would prefer would be an *idealized* language for expressing mathematical thoughts — a compact, precise and purely symbolic notation in which the mechanisms of ordinary language have been stripped down to their bare essentials, and from which the ambiguities and imperfections of human languages have been eliminated.

This concept of an ideal 'language of thought' goes back at least as far as Gottfried Leibniz (1646-1716), the brilliant and wide-ranging mathematician, philosopher and lawyer perhaps best known today as the co-discoverer (with Isaac Newton) of integral and differential calculus. Leibniz had a keen

appreciation of the power of a good mathematical notation (his symbolism for calculus is still in use today), and this interest led him to the grandiose conception of a *characteristica universalis* (roughly a 'universal character system') — a precise, symbolic notation in which not just mathematical ideas, but *all* human ideas and knowledge, could be clearly and unambiguously expressed. His hope, moreover, was that such a notation could then form the basis for a *calculus ratiocinator* (or 'reasoning calculus') — a set of rules for manipulating symbols, rather like those of school algebra, which would allow logical reasoning across the whole spectrum of human affairs to be carried out precisely and reliably by means of simple calculation. Leibniz had high hopes indeed for such a system:

> If we had it, we should be able to reason in metaphysics and morals in much the same way as in geometry and analysis.…
>
> If controversies were to arise, there would be no more need of disputation between two philosophers than between two accountants. For it would suffice to take their pencils in their hands, to sit down to their slates, and to say to each other … 'Let us calculate!'

An idea like this makes even Hilbert's decision problem seem pretty small beer. Needless to say, Leibniz didn't get very far towards realizing this dream in his lifetime. Even as regards the much humbler goal of providing a symbolic language for *mathematical* thoughts, his idea was centuries ahead of its time, and it had to wait until 1879 for the logician Gottlob Frege to provide the key insights that made this goal attainable. In a short booklet entitled *Begriffschrift* (which translates roughly as 'Concept Notation'), Frege effectively laid the foundation for modern mathematical logic. The Leibnizian spirit of the work is evident in the book's subtitle: 'A formula language, modelled upon that of arithmetic, for pure thought'. In essence, Frege's notation did turn to be powerful enough to express all statements of ordinary mathematics, and it forms the basis for what is today called *predicate logic*. Explaining the notation of predicate logic is one of the main purposes of this chapter.

The glyph language

The language we introduced in the story is a particular version of predicate logic used for expressing properties of (Real or Whole) numbers. We introduced this language in two stages: first, a simple language of algebra, which may seem not too far removed from things you did at school; and second, an extension of this with some special logical symbols which I'm

calling *glyphs* (this isn't standard terminology), and which massively extend what the language is capable of expressing.

As we saw, the language comes with certain 'grammatical rules', which amount to a precise definition of which sequences of symbols are 'legal' in the language and which are not. If you know a bit about how computer languages work, some of the ideas here might well be familiar to you. There's no need to go over all the rules again in detail here, but let's just recap on the main points.

The starting-point for the language is the choice of a suitable *alphabet* — that is, a finite repertoire of distinct symbols from which statements will be built up. Here's the alphabet I have chosen for our language:[1]

a b c d e f g h i j k l m n o p q r s t u v w x y z ′
0 1 2 3 4 5 6 7 8 9 + × ( ) = < ≤
¬ & ∨ ⇒ ∀ ∃

The ten symbols 0,1,…,9 will be called *digits*, and the twenty-six symbols a,b,…,z will be called *letters*. Actually, this is a rather generous alphabet, which I've chosen with the convenience of ten-fingered English-speakers in mind — you could get away with quite a bit less and still have a language of the same power. Anyway, the idea is that statements in the language are built by clumping symbols together in larger and larger combinations according to certain rules. First, we define certain clumps of symbols known as *numerals* (such as 341), bearing in mind that such a 'clump' might consist of just a single symbol (such as 5). We also define clumps called *variables*: a variable can be just a single letter, such as y, or a letter followed by one or more copies of the dash symbol, such as x″ (this is just a simple device to ensure that we'll never run out of variables). Numerals and variables are the simplest examples of what are called *terms*. But we can also clump terms together to form bigger terms such as (y + 5) or (x″ × 341).

Continuing our clumping process further, any two terms can be combined with one of the symbols $=$, $<$, $\leq$ to yield what I'll call a *basic formula*, such as (y + 5) = (x″ × 341). Finally, basic formulae can themselves be clumped together using the *logical* symbols $\neg$, &, $\vee$, $\Rightarrow$, $\forall$, $\exists$ as described in the story, to yield the full class of *formulae*.

Although in the story I presented these clumping rules in a 'wordy' rather than a mathematically crisp way, I hope you can see that the rules themselves are quite precise and clear-cut in nature.[2] The key point is that *only* those glyph sequences that can be constructed via these rules are to be considered as 'grammatically correct' formulae in the glyph language — and this means we have a precise, rigorous definition of what is a legal formula and what isn't. Indeed, for the language as I've described it, it wouldn't be too hard to program a computer to recognize whether a given string of symbols was a

legal formula, and — if it was — to analyse it in terms of the various clumps from which it was built up. Having a mathematically precise definition of our language will be an essential foundation-stone for our project, not least because this will make it possible to *prove* things about our language with mathematical rigour.

Another important point to notice about these grammatical rules is that they make no reference at all to the intended *meaning* of the various symbols, or to whether they even have a meaning. Just as in the game of Dominoes, there are certain rules for how pieces may be placed together, but we don't think of the pieces as 'meaning' anything, so our language has a certain rules for plugging symbols together which could be grasped by someone who knew nothing of their meaning. Indeed, a computer that checked whether a given sequence of symbols was legal would be oblivious to the meanings we usually associate with them. This is a significant point, because one of the main questions we'll be asking later is: Which aspects of mathematics can be adequately accounted for without appealing to this rather difficult and elusive notion of 'meaning'. We will return to this in Chapter 5.

The precision and mechanizability of our language does come at a cost. For most human readers, extracting sense from of a symbolic formula is usually harder than understanding a statement written partly in English. What's more, you will have noticed that our formulae tend to be cluttered with a lot of *brackets*, which might sometimes seem to hinder rather than help. The brackets are there because they provide a simple way of ensuring that every legal formula has a single, unambiguous structure. For instance, if our terms didn't involve brackets, what would we make of a term like $3 + 4 \times 5$? Was this constructed by applying '$\times$' to $3 + 4$ and 5, or by applying '$+$' to 3 and 4×5? There are various ways of responding this problem, but the simplest way — and the one I've adopted — is just to insist on so many brackets that ambiguities never arise: we have to write either $((3 + 4) \times 5)$ or $(3 + (4 \times 5))$. Having said that, in later chapters I'll sometimes be a bit more sloppy, omitting brackets that should officially be there to improve readability. If you are a purist, you can amuse yourself by putting them back in.

There's also one little technical point that I didn't fully cover in the story. I said earlier that a mathematical statement should be *self-contained*: in particular, it shouldn't involve any 'free-floating' variables whose meaning or role isn't specified. For example, the formula $n < 5$ isn't self-contained, because of the free variable n; but $(\exists n \;\; n < 5)$ is, because the idea is that the sequence $\exists n$ somehow 'introduces' the variable n. This is easy to understand intuitively if we think ahead to what our symbols will be used to mean: '*there exists* a number n such that $n < 5$'. This is self-contained because the 'there exists' introduces the 'n' and explains its role within the statement. But for the time being, in the spirit of our symbolic approach, we want to supplement our

existing grammar rules with a definition of 'self-contained formula' that makes no reference to the intended meaning of the symbols.

Let's see how we can do this. The reason the formula $(\exists n \;\; n < 5)$ is 'good' while $n < 5$ by itself is 'bad' is that in the first case, the variable n occurs inside a subformula of the form $(\exists n \ldots)$, while in the second, it doesn't. Or to give a more complicated example, look at the formula

$$(\exists n \;\; n < 5) \;\; \& \;\; (\forall m \;\; n < m)$$

Here the first two appearances of the variable n are fine, since they occur within the subformula $(\exists n \;\; n < 5)$. And the two appearances of m are fine, since they both occur within $(\forall m \;\; n < m)$. But the third appearance of n is bad, because there's no $\forall n$ or $\exists n$ that 'introduces' it. This suggests that we should make a rule that says:

> Every appearance of the variable n has to occur inside some formula of the type $(\forall n \ldots)$ or $(\exists n \ldots)$.

Of course, we want this to apply not just to the variable n, but also to m and to any other variable we might use. So let's say that a formula is *self-contained* if it obeys the following *No-Free-Variable Rule*:

> Every appearance of a variable occurs within some formula $(\forall\ldots)$ or $(\exists\ldots)$ in which the leading $\forall$ or $\exists$ is immediately followed by that same variable.

A formula which is self-contained in this way will from now on be called a *glyph statement*. Although we've explained the *purpose* of the No-Free-Variable Rule with reference to what formulae are intended to mean, it's clear that the rule itself is something you can check purely symbolically. For example, even without trying to 'understand' the complicated formula below, you can check (if you like) that it obeys the No-Free-Variable rule: every appearance of n occurs within a $(\forall n \ldots)$ subformula, every p occurs within a $(\exists p \ldots)$ subformula, and likewise for all the other variables.

$$\begin{gathered}(\forall\ n\ \ ((2 < n\ \ \&\ \ (\exists\ m\ \ (m \times 2) = n))\ \ \Rightarrow\ \ (\exists\ p\ \ (\exists\ q\ \ ((p + q) = n\ \ \& \\ ((1 < p\ \ \ \&\ \ \ (\forall\ a\ \ (\forall\ b\ \ \ ((a \times b) = p\ \ \Rightarrow\ \ (a = 1 \vee b = 1)))))\ \ \& \\ (1 < q\ \ \ \&\ \ \ (\forall\ c\ \ (\forall\ d\ \ \ ((c \times d) = q\ \ \Rightarrow\ \ (c = 1 \vee d = 1)))))))))))\end{gathered}$$

Putting the meaning back

That completes what we need to say about the rules of the glyph language. We've already stressed that these rules are totally blind to the intended meaning of the symbols, or to whether they have a meaning at all. So the grammar rules of the language, taken by themselves, give no clue as to whether '∀' is supposed to mean 'for all' or 'there exists', nor even whether the symbol '5' is supposed to refer to the number we usually call 'five' or the one we call 'seven'.

This 'bleaching out' of the intended meaning happens as a natural result of formulating the grammar rules in a purely symbolic way. And of course, this will be exactly what we need if we're wanting to see how far we can get in mathematics *without* any concept of 'meaning'. But for other purposes, a concept of meaning might play an important role. For instance, in our story, Tarski's Decider takes a glyph statement, and works out whether or not it's *true* (as a statement about Real numbers). But describing a statement as 'true' presupposes that it means something: if a glyph statement were just a meaningless gobbledegook of symbols, what could it possibly mean to ask whether it was true?

This leads us to the idea that if we want to do anything that relies on a concept of 'meaning' or 'truth' for glyph statements, we shall have to *define* such a concept. In other words, having rigorously excluded all reference to meaning in the way we set up our grammar rules, we now want to put it back again. Along with this comes the idea that we are, in principle, free to define the meaning of our language in any way we choose, since the formal grammar of the language doesn't impose any particular meaning on us.

In fact, we're even at liberty to introduce *several* concepts of meaning or truth serving different purposes. This was the point of the spectacles in our story: the Real and Whole number spectacles represent two possible notions of 'meaning' for the exactly the same formal language.

How can such concepts of meaning or truth be defined? There are various ways in which this might be done, but for our present purposes, a rather simple approach (sometimes going under the rather grand name of 'Tarski's definition of truth'[3]) will suffice. In essence, the idea is to assign 'meanings' or 'values' first to the very smallest components of the formula (such as the numerals), then use these to work our way up to larger and larger components, until eventually we arrive at a truth value ('true' or 'false') for the whole formula.

As an easy example, suppose we have a string of symbols such as:

$$(((3 + 4) + 5) = 7 \ \vee \ \neg 6 = 9)$$

and we want to know whether this should be considered true or false. The idea is that we can assign a numerical value to each *term* within this string, and a truth-value (true or false) to each *formula* within it, by means of a bunch of general rules such as the following:

1. The value of any numeral is the Whole number it usually refers to: for instance, the value of the glyph sequence '53' is the number fifty-three.
2. The value of a term (T + U) is the value of T plus the value of U.
3. A formula T = U is true if and only if the values of T and U are equal.
4. A formula ¬ P is true if and only if P is not true.
5. A formula P ∨ Q is true if and only if P is true or Q is true (or both).

It's not hard to check that, by successively applying these rules to the various constituents of the formula above, we can arrive at the answer: the formula (((3 + 4) + 5) = 7 ∨ ¬ 6 = 9) is *true*.[4]

There will also be some similar rules for ×, <, ≤, & and ⇒. Strictly speaking, Rule 1 above should be expanded to spell out just *how* a sequence of decimal digits determines a Whole number, but we won't bother with this level of detail here.

Something about these rules might strike you as odd. All they seem to tell us is that 3 means 'three', + means 'plus', ∨ means 'or', and so on. In other words, they clearly presuppose that we already understand what 'three', 'plus' and 'or' are supposed to mean! So the rules aren't offering us any actual *explanation* of these concepts — we're supposed to understand them already. They are merely bridging the gap between the level of *glyph statements*, which we want to think of as pure symbol sequences and in themselves 'meaningless', and the level of *mathematical statements* which we ourselves already understand, or think we understand. Not that the rules are redundant: after all, we might have chosen instead to specify that ∨ means 'and'. But if we were hoping for some deep philosophical insight into the essence of these concepts, then this clearly isn't it. Whether it's possible to give some more fundamental analysis that explains the actual meanings of primitive concepts like 'plus' and 'or' is something we'll explore in later chapters.

We still haven't quite finished yet. This example was a bit too simple, because there were no *variables* involved. I said that the idea is to assign a numerical value to each term involved. But what would be the value for a term like (p + q) ? Obviously, this doesn't have any absolute numerical value — it depends what values we choose to assign to the variables p and q. To take account of this, we should therefore modify our rules so that they define the value of a term or formula *relative to* some choice of values for the variables involved. (For further details, see Appendix B.)

Once we've set this up, we can also give the rules for our last two logical constructs, ∀ and ∃. Assuming it's the Real number interpretation we're wanting to define, the rules for these will look roughly like this. (I'll number these as 11 and 12 in recognition that our earlier list omitted a few rules):

11. A formula (∀x P) is true if, *for every* Real number *r*, the formula P comes out true when we give x the value *r*.
12. A formula (∃x P) is true if, *for some* Real number *r*, P comes out true when we give x the value *r*.

These capture the expected meanings of ∀ as 'for all' or 'for every', and of ∃ as 'there exists' or 'for some', on the understanding that we're working with Real numbers. The Whole number interpretation can be defined similarly.

It's worth pointing out that these last two rules are a bit different in character from the ones we gave earlier. They don't actually give a way of *calculating directly* whether or not a formula like (∀x P) is true, because there are infinitely many possible choices for the number *n* that we'd have to consider. Rather, they simply give a mathematical definition of *what it means* for such a formula to be considered true. Indeed, given that our language can be used to express quite complex mathematical statements, we wouldn't expect any very simple recipe for just calculating their truth-value. Doing that is typically a hard problem, and — when it's possible at all — is the business of the *decision algorithms* which we'll come onto shortly.

At this point, you might want to test your understanding by revisiting the three-line glyph statement given at the end of the last section, and seeing whether you can figure out what it says when viewed through the *Whole number* spectacles. The answer is given in the endnotes.[5] Or for those who are wanting to train as fully qualified glyph wizards, the book's website offers a supplement entitled *Further tricks in glyph language*.[6]

Truth tables

One last thing before we leave the subject of truth definitions. I said earlier that our definitions shed no real light on the meanings of our symbols, because they rely on our prior understanding of the meanings of 'plus', 'or' and so on. But actually, in the case of the logical symbols, it's possible to do a little better than this, or at least to offer a slightly different perspective. The idea here is that as soon as we know whether P is true or false and whether Q is true or false, we're in a position to say whether the formula P ∨ Q is true or false; and likewise for the formulae P & Q, P ⇒ Q, and ¬ P. This suggests that we can draw up *tables* in which one can simply look up the truth-values of these formulae, given the truth-values for P and Q.

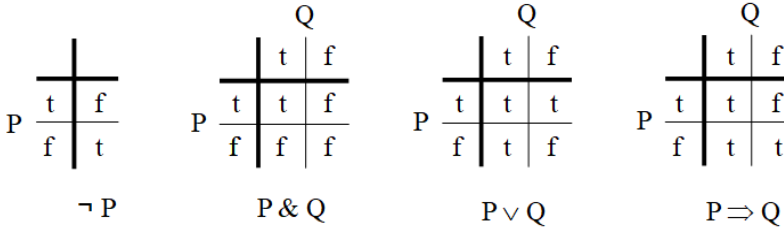


Figure 1: Truth tables for ¬, &, ∨, ⇒.

So here, rather than simply relying on your prior understanding of the intended meanings of 'not', 'and', 'or' and 'implies', I've been more explicit in saying exactly what the truth-values of such formulae are supposed to be, given the truth-values of their components. This idea is at the basis of what is now called *Boolean logic*, in honour of its pioneer George Boole (1815-1864), although it was W.S. Jevons and C.S. Peirce who really nailed down the form of the logic that we use today. These ideas turned out to have an impact that their originators can scarcely have dreamed of: Boolean logic and truth tables are fundamental to the way today's computers work.[7]

In the light of what we've said so far, the tables for ¬, & and ∨ should come as little surprise, but the one for ⇒ deserves a bit more explanation. On the one hand, it's not too hard to see what the single 'false' entry in the table is doing there: if, in fact, P is true but Q is false, then it would certainly be wrong to say that 'P implies Q', or 'if P then Q'. But what might be puzzling is that we're taking the formula P ⇒ Q to be true *in all other cases*. Certainly this doesn't fit with the way we tend to use 'if … then …' in ordinary speech: surely it would be a bit eccentric to claim that 'if unicorns exist, then there are no such things as unicorns'? The fact is that the way we ordinarily use 'if … then …' is much more subtle than can be accounted for by a humble truth table. However, for the purposes of classical mathematics, our simple truth table turns out to be quite adequate. This way of understanding the meaning of 'implies' might seem odd at first, but one does get used to it.[8]

Would it be possible to give truth tables like these for the other logical operators, ∀ and ∃? Sadly not: to calculate the truth-value of (∀x P), we in principle need to know the truth-value of P for *every* possible value of x, so we'd need a truth-table with infinitely many dimensions. What's more, I wouldn't be able to tell you what this infinite-dimensional truth-table was like without employing the notion of 'for all' or 'for every'. So in this case, I need

to rely on your prior understanding of the concept of 'for every' (as in our earlier clause 11) for the interpretation of the symbol $\forall$.

The elementary particles of logic

Let's now stand back and take a look at what we've done, and what we have and haven't achieved. First, it will be helpful to clarify the distinction between what we'll call the 'logical' and 'non-logical' parts of the language.

The glyph language that we've described is obviously designed for talking about numbers (whether Real or Whole): we've built into it not only ways for referring to particular numbers, such as 5, but symbols for some of the fundamental operations and relationships that are relevant to numbers, such as $+$ and $\leq$. If we wanted a similar glyph language for, say, Euclidean geometry, we would want to consider a different set of fundamental relationships — the idea of a point lying on a straight line, the idea of two distances being equal, and so on. Or if we wanted a language for more advanced kinds of mathematics, it might be useful to consider 'being a member of a set' as a fundamental relationship. Even so, all these languages would have a lot in common: in particular, the purely *logical* ingredients would stay the same, since the ideas represented by 'and', 'or', 'for all' and so on are equally applicable whether we are thinking about numbers, points or sets. In other words, we can think of the ingredients of our language as being of two kinds: the *non-logical* ingredients, which are those specific to the particular (in this case mathematical) subject matter which the language is designed to talk about; and the purely *logical* ingredients, which are in some way general and universal, and will be equally applicable to *whatever* we want to talk about. The great contribution of Frege's predicate calculus was, in essence, to identify a small set of logical ingredients that provide the basic infrastructure of a 'language of thought' for mathematics, no matter what kinds of mathematical objects the language is meant to be about.

In our glyph language, the ingredients that we're calling 'logical' are the concept of a *variable*, along with the following logical constructs:[9]

$\neg$ (not) $\quad$ $\&$ (and) $\quad$ $\vee$ (or) $\quad$ $\Rightarrow$ (implies) $\quad$ $\forall$ (for all) $\quad$ $\exists$ (there exists)

In fact, Frege's original calculus was even more minimalist: he took just $\forall$, $\&$, $\neg$ as the basic constructs, and showed how $\exists$, $\vee$, $\Rightarrow$ could be expressed in terms of these.[10] In this book, for various reasons, I shall prefer to treat all six operators as primitive. But in any case, what is really striking is that in terms of purely *logical* machinery, such a small repertoire of 'elementary particles' is sufficient to express any statement in mathematics from before Frege's time or since.

Beyond the Big Six?

That's not to say that these basic constructs exhaust *everything* that might reasonably be considered under the umbrella of 'logic'. Indeed, once we move beyond the pure realm of mathematics into other areas of human thought and language (as Leibniz wanted to do), we very quickly discover things that can't readily be expressed using this small handful of primitives — and to capture these, logicians and linguists have been led to adopt a wealth of other logical gadgetry going beyond the six operations we've mentioned. Let's look very briefly at some examples.

Suppose, for instance, we agree to include a non-logical formula symbol R in our language, with the intended meaning 'It is raining'. Then we'd already know how to express symbolically the statement 'It is not raining' ($\neg$ R) or the statement 'Either it's raining or it's not raining' (($R \vee \neg R$)). But how would we express 'It *was* raining', or 'It will be raining', or 'It might be raining', or 'Emily thinks Anna knows it is raining'?

Such statements seem to call for other kinds of logical operators, taking us into the realm of *tense* and *modal logics*. In tense logic, for instance, we extend our repertoire of logical operations $\neg$, &, $\vee$, $\Rightarrow$, $\forall$, $\exists$ with others such as P (meaning 'it was once true that …') and F ('it will at some time be true that …'). So the statement 'it was raining' can now be expressed by PR. Likewise, in modal logic, we introduce operators such as $\frac{1}{2\pi}$ and $\Diamond$, typically with meanings along the lines of 'it must be that …' and 'it could be that …' respectively, although there's room for a lot of choice in how exactly these symbols are interpreted. A major advance was made in the 1970s by Richard Montague, who showed that a system with the usual six operations plus four modal operators could account for a wide range of subtle linguistic phenomena, even including statements that appear to involve non-existent entities, such as 'Anna thinks Emily is looking for a unicorn.'[11]

But even Montague's theory has its limits. Perhaps its greatest weakness is that it only tells us how assign logical forms to individual sentences taken in isolation, whereas in actual language use, the meaning of a sentence will very often depend on the context in which it's used. For instance, the meaning we take from a statement like 'It's under the sofa' will depend on what the 'It' refers to, which will be determined by the context. So to account for these aspects of meaning, we need to widen our field of view to take in not just individual sentences, but entire *discourses*.

It turns out that standard predicate logic is not very good at dealing with this sort of thing. Suppose, for example, that we're talking about a set of ten marbles, and I tell you: 'Just one of the marbles isn't in the bag'. Taking variables to refer to marbles, we might formalize this as:

$$(\exists\, x \;\; (\neg\, \mathrm{InBag}(x) \;\&\; (\forall\, y \;\; (\neg\, y = x \;\Rightarrow\; \mathrm{InBag}(y)))))$$

But now suppose I follow up my statement with 'It's under the sofa'. What does the 'It' refer to in this context? Intuitively, to 'the marble that isn't in the bag', whose existence was established by the previous sentence. But how would we translate that into formal logic? We somehow want a formula that 'points to' the variable x that occurs inside the previous formula, and predicate logic doesn't offer any good way of doing this. In order to give a satisfactory logical account of this kind of thing, linguists have developed further frameworks such as *Discourse Representation Theory*, which takes seriously the idea that the very act of saying something *changes* what it's possible to talk about next.

So for many different reasons, theoretical linguists have been led to explore new approaches to logic going well beyond our Big Six. But there's a further twist to the story. In order to clarify the intended meaning of this weird and wonderful new gadgetry, linguists have also found it useful to look at ways of explicating it in more familiar terms via a Tarskian style of interpretation. For instance, to explain the tense operator P, we might introduce the simple idea of a time-line, and say that PR is to be interpreted as 'there exists a time in the past at which R holds'. Or to explain the modal operator $\frac{1}{2\pi}$, we might introduce the idea of a *possible world* (whatever one of those might be), and say that $\frac{1}{2\pi}$R means 'in all possible worlds, R holds'. But what has happened here? In effect, we've *translated* the meanings of our new gadgets, P and $\frac{1}{2\pi}$, into statements that can be expressed using just the old ones, $\exists$ and $\forall$ (at the cost of introducing the additional notions of 'points in time' and 'possible worlds'). So it may be that our Big Six, after all, do play some fundamental role at the core of all thinkable thoughts.

All in all, there seems to be a great deal left to explore in this fascinating territory at the borderland between language and logic. In this book, though, we are mainly thinking about the logic that's required to do mathematics, so we shall be concentrating almost entirely on bog-standard predicate logic.

Finally, I ought to emphasize that even as far as mathematics is concerned, the mere definition of a suitable language is only half of the story. For Leibniz, as we saw, the project of formalizing human thought was to involve two stages: designing a suitable symbolic notation, and providing a set of rules for logical reasoning by pushing the symbols around in certain ways. What we've done so far (for a certain portion of mathematics) corresponds to just the first of these stages. In Chapter 5 we'll come on to the second stage — the rules which allow us to mimic the steps in a logical argument by pure symbol-shunting.

Having said that, even the first stage opens the door to some interesting possibilities, as we shall now see.

Decision algorithms

Let's return to Hilbert's question, which we mentioned at the end of Chapter 2. Is there some purely mechanical method or algorithm for telling whether a given mathematical statement is true or not? Now that we're able to express many mathematical statements as glyph statements, and we also have a definition of 'truth' for glyph statements, this is a sensible question to ask, because a glyph statement, being just a sequence of formal symbols, is exactly the kind of thing that a mechanical procedure could reasonably be expected to work on.

Actually, this question needs to be asked separately for each glyph language we're interested in, and indeed for each interpretation of such a language. For instance, we saw in our story that that the answer to Hilbert's question is 'Yes' if we limit our attention to Real number glyph statements (this was first shown by Tarski in 1930), but the corresponding question for Whole number glyph statements was left open. A mechanical procedure that works on some class of mathematical statements and classifies them into 'true' and 'false' is called a *decision algorithm* for that class of statements.

Tarski's decision algorithm for Real number statements is actually a rather complicated example. To get the feel of decision algorithms, it's best to start with something much simpler, namely with what is called *propositional logic*. Compared with what we've been considering so far, propositional logic may seem quite crude, because instead of starting from formulae like $(x + y) = 5$ or $z < (2 \times t)$, we simply use *formula variables* such as P, Q, R as our basic formulae, and build up other formulae from these using $\neg$, &, $\vee$ and $\Rightarrow$. For example:

$$(\neg (P \,\&\, Q) \;\Rightarrow\; (\neg P \vee \neg Q))$$

You can think of each formula variable as standing for some simple statement that could be true or false, like 'It is raining' or 'Emily is asleep', though we don't attempt to represent the internal structure of such statements in any way. We also don't include the operators $\forall$ and $\exists$ in our propositional logic.

What would it mean to give a decision algorithm for this logic? Obviously, whether a propositional formula is true may depend on whether its formula variables are taken to be true or false; but the natural question to ask here is whether a given formula is *always* true — that is, whether it's true *whatever* we take the truth values of the variables to be. For example, the formula above can be paraphrased as 'If it's not the case that both P and Q are true, then either P is not true or Q is not true' — and it's not hard to see that this statement itself *must* be true, irrespective of the particular truth-values of P and Q.

Is it possible to give a purely mechanical procedure for telling whether a given propositional formula is always true? In fact, this is quite simple: all we have to do is to consider, in turn, all possible ways of giving truth-values to the formula variables in question, and for each of these, to calculate whether our formula is true or not. For instance, the formula above has just the two variables P and Q, so there will be four possibilities to consider:

$$P \to t, Q \to t \qquad P \to t, Q \to f \qquad P \to f, Q \to t \qquad P \to f, Q \to f$$

In each of these four cases, we can simply apply the truth-tables we gave earlier to work out the truth-value of the complete formula. In general, for a formula with n distinct formula variables, there will be 2^n cases to consider. If our formula comes out true in all of these cases, we know that it is always true. This gives us a simple decision algorithm for this logic.

Actually, decision algorithms for propositional logic are quite big business nowadays. People who design computer chips and other electronic circuitry routinely find themselves working with propositional formulae involving thousands of variables, and computer implementations of decision algorithms play an essential role in checking that their designs are correct. Not that the simple-minded algorithm we described above is fit for such purposes: for a formula with 1000 distinct variables, there would be $2^{1000} \approx 10^{301}$ cases to consider, so there's no chance of checking all of these separately. But in the last two decades, remarkable progress has been made in devising much cleverer algorithms, which in practice work well even for propositional formulae running to millions of symbols, with tens of thousands of distinct variables. The computer programs that implement these algorithms are known as *SAT solvers*, and there is a lively and competitive field dedicated to making them as efficient as possible. (Here 'SAT' refers to the 'satisfiability problem', which is an alternative way of formulating the decision problem as I've presented it.) Nowadays, no package for computer-aided circuit design is complete without a powerful SAT solving engine, and SAT solvers have also found applications in artificial intelligence and even in pure mathematics.[12]

Of course, propositional logic is extremely simple by comparison with the theory of the Real numbers and other theories that we'll consider. But our brief glance at propositional decision algorithms perhaps helps us to appreciate what's so remarkable about Tarski's algorithm. In the case of propositional logic, our 'variables' could take just two values — true and false — so it was clear we could tell if a formula was always true just by checking a finite number of cases (2^n if there are n distinct variables). In our Real number language, however, the variables refer to Real numbers, and there are infinitely many of these, so it's already clear that we can't decide the truth of a statement just by exhaustively checking all possible instances. Some cleverer idea is required.

It would go beyond the scope of this book to explain just how Tarski's solution worked, except to say that it combined two ingredients. One was a piece of 19th century mathematics: an algorithm due to Charles-François Sturm for counting and locating solutions to polynomial equations. (This corresponds to what Anna's Equation Solver did in the story.) Sturm's algorithm really belonged to the realm of algebra and calculus rather than logic, but Tarski vastly extended what it could achieve by adding a logical technique known as *elimination of quantifiers*. This is what made it possible for the algorithm to cope with complex glyph statements rather than just simple equations.

What does Tarski's result mean in practice? You might suppose it meant that Real number questions expressible in glyph language would now all be viewed by mathematicians as 'trivial', since they can be answered just by turning the handle (or by running a computer program). Actually, it's not quite as simple as this. Although Tarski's result assures us that *in theory* all such questions can be answered by letting a computer program run for long enough, that's not the same as saying this can be done *in practice*. In the story, I symbolically represented Tarski's algorithm as taking quite a long time to run, but the timescales I suggested were not realistic: even for the simple examples we gave, the time needed to run Tarski's original algorithm on today's computers would probably exceed the present age of the universe! That said, Tarski's method has since been significantly improved on, and there are now decision algorithms which are practically useful for handling smallish problems (say one-line glyph statements) within reasonable time. But it remains the case that for more complex glyph statements, mathematicians may well be better off resorting to their imagination and intelligence if they want an answer.

In fact, we even have the curious situation that there are some interesting mathematical problems whose answer is currently *unknown* — even though they can in principle can be solved by Tarski's algorithm. One of these problems is so elegant that I can't resist telling you about it.[13]

The kissing spheres problem

We'll start with an easy question in two-dimensional geometry. Suppose you are given a circle. How many other circles of the same size can be arranged to 'kiss' the circle (that is, to touch it at just a single point) without overlapping one another? A moment's reflection shows that the answer is 6 — there clearly isn't room for any more (see Figure 4).

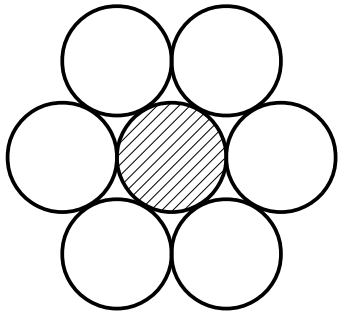


Figure 2: A circle kissed by six others of the same size.

Now let's go up to three dimensions. You're given a *sphere*, and your problem is to arrange as many spheres of the same size around it as possible, touching the original sphere at a single point but not overlapping. It isn't too hard to arrange 12 other spheres in this way (if you know what a dodecahedron looks like, this will give you one way of doing it). But there's still quite a lot of spare room left over, and you might wonder whether, with some clever rearrangement, you could squeeze in a thirteenth. In fact, this very question was debated in 1693 by Isaac Newton (who thought you couldn't) and David Gregory (who thought perhaps you could). The issue wasn't definitively settled until 1953, when Schütte and van der Waerden showed that Newton was right: 12 kissing spheres is the maximum.

What happens if we go up to *four* dimensions? This might seem a rather pointless question, given that in practice we only have three physical space dimensions to play with, but let's ask it anyway. Don't worry if you can't 'visualize' a four-dimensional space (I can't either, though there are apparently some people who can). All you need to know is that such a concept makes good mathematical sense, and that corresponding to circles in two dimensions and spheres in three dimensions, there is a natural concept of a *hypersphere* in four dimensions. So, given a hypersphere in 4-space, how many identical hyperspheres can be arranged to kiss it? It was shown by Oleg Musin in 2003 that the so-called 'kissing number' for four dimensions is 24.

So what about five dimensions? Here, we don't have an exact answer: at the time of writing, all we know is that at least 40 and at most 44 kissing hyperspheres can be fitted in. Within this range, the exact value of the kissing number is currently an unsolved problem: it might be 40, 41, 42, 43 or 44.

But the curious thing is that we *do* know a method that's guaranteed to solve this problem! This is because the problem can be quite easily expressed in terms of Real number glyph statements. For example, it's quite easy to knock together a glyph statement that means '41 kissing hyperspheres in 5-space are possible', along with similar glyph statements for 42, 43 and 44. (If you know some coordinate geometry, you may well be able to see how to

construct such glyph statements — although they'd be rather long to write out.) So all we'd need to do in order to answer this question is to run Tarski's Decider (or something equivalent) on each of these four glyph statements in turn. If the results were 'true, true, true, false', for instance, we'd know that the maximum number of kissing hyperspheres was 43. So we're in a frustrating and tantalizing situation: we *know* a completely automatic method which is guaranteed to lead to a solution to this problem — only we're unable to carry out the computation owing to its super-astronomical length.[14] The answer, so to speak, is sitting up there on a high shelf, *and we know exactly where*, but it's beyond our reach. Of course, that doesn't mean that we'll *never* know the answer to this question — only that we're unlikely to find it just by grinding our way through the algorithm. The situation is analogous for kissing numbers in most higher dimensions.[15] Almost certainly, this is a case where it's better to exercise ingenuity and creative thought, perhaps supplemented by some use of computer power.

Computer proofs and decidable theories

As we've seen, Tarski's method and its descendants are impractical for answering large, complex questions about Real numbers. But that doesn't mean they have *no* practical value. As we mentioned above, these days we have methods which are effective for solving simple 'one-line' problems of this kind using a computer, and that's already enough to be useful and interesting. For instance, there's a sizeable community of people working on using computers to help human mathematicians construct proofs. The idea here is that the human mathematician should supply the clever ideas and the key steps in the proof, while the machine takes care of the boring details, and also checks that the human's proof steps are correct. Now, it could be that certain steps in the proof come down to checking whether some Real number statement is true. If they do, and if the statements are simple enough, then these steps can indeed be delegated to the machine. The upshot of all this is that we can acquire confidence in the correctness of very long and intricate proofs for which it would be virtually impossible for a human to check every detail by hand (such proofs often arise, for example, when we try to show that the design of some piece of hardware or software is correct). It's worth emphasizing that none of this would be possible without a precise symbolic language for mathematical statements — such as our glyph language — which can be manipulated by humans and machines alike.

As we've seen, Real number glyph statements represent an interesting class of mathematical problems, but they certainly don't exhaust the whole of mathematics. But now that we have one example of a *decidable* class of mathematical statements (that is, one for which a decision algorithm exists),

it's only natural to ask what other such classes there might be. Are there other areas of mathematics within which the truth or falsity of a statement can be determined by a purely mechanical algorithm (either in principle, or maybe in practice as well)? This is the sort of thing that the machine-assisted proof people are interested in, since they want to know which kinds of proofs (or steps within a larger proof) could in principle be left to a machine.

Actually, many other examples of decidable classes of statements are known, showing that Hilbert's idea is quite viable if restricted to certain suitable *theories* (roughly speaking, certain well-defined 'areas of mathematics'). Some of these deserve a brief mention here, though they won't play a major role in the rest of the book.

First, the theory of *Euclidean geometry* turns out to be decidable. That is, if we set up a suitable version of our glyph language for ruler-and-compass style geometry (as we suggested a few pages back), one can show that there's a Decider for statements of this language: an algorithm which tells whether a given statement is true. Actually, it's not hard to see roughly why this is so: using the idea of *coordinates*, any point in the Euclidean plane, say, can be represented by a pair of Real numbers — and in fact, *any* statement in the geometry language can be easily and automatically translated to a glyph statement in Real number language. So we can simply perform the translation, and use Tarski's algorithm (or similar) on the resulting statement. A very similar trick can also be used to show that the corresponding theory of *Complex numbers* is decidable: a Complex number can be represented by two Real numbers, so once again, statements about Complex numbers can be translated to statements about Real numbers that we already know how to solve.

An interesting question is what happens if we extend the Real number language by adding in the operation of *exponentiation* (x^y, or 'x to the power y') alongside those of addition and multiplication. The situation here is intriguing. It's believed that this larger class of statements is indeed decidable — not only that, but there's even a specific known algorithm which is believed to be a decision algorithm for such statements. However, as things stand today, there's no complete proof that it *is* a correct decision algorithm.[16] In other words, we *do* have what is very probably a Decider for this theory — we're just not completely sure that it is one, because we don't have the master medallion to go with it.

We've concentrated so far on decision algorithms for Real number statements. But what about *Whole* numbers? At the end of our story chapter, we left Emily wondering whether a decision algorithm for Whole number glyph statements existed. We shall learn the answer to this in the next chapter, but in the meantime, it's worth noting that the answer is certainly 'Yes' if we cut down to a slightly smaller glyph language. One way to do this is just to drop the '×' operation from the language (but keep +, < and everything else).

The resulting fragment of mathematics is known as *Presburger arithmetic*, and it's known to be decidable. (For smallish examples, it's also decidable *in practice*, which is good news for the machine-assisted proof people.) Alternatively, you could keep '×', but drop '+' (along with < and ≤). This fragment is (not too far from) what is known as *Skolem arithmetic*, and it too is decidable.

These facts are summarized by Figure 5 below, in which the solid circles represent various areas of mathematics which we have seen so far to be decidable. The arrows indicate that questions about Euclidean geometry and Complex numbers can be solved by translating them into Real number glyph statements.

The main remaining question, then, is whether the *full* class of Whole number glyph statements is decidable — that is, whether we still have decidability when we allow statements containing both '+' and '×'. We shall come back to this in the next chapter.

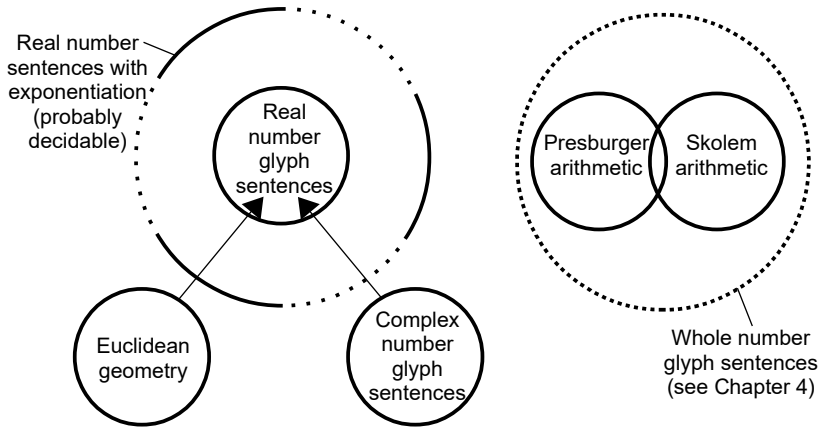


Figure 3: Some mathematical theories and their relationships.

Undecidable theories

So far, we've seen that quite a few interesting theories (that is, areas of mathematics) are decidable, but we haven't yet established for sure that there's even such a thing as a theory that *isn't* decidable. If undecidable theories do exist, why have I only told you about decidable ones? In fact, there's a good reason for this: at this stage in the book, we don't even have a precise way of saying what it means for a theory to be 'undecidable'. Let me explain this further.

There's a basic asymmetry between the concepts of decidability and undecidability which it's important to grasp. Remember that to say a class of

statements is decidable is to say that there's an algorithm — a purely mechanical procedure or recipe — for telling whether a statement in the class is true or not. So to show that a theory is decidable, all you have to do is to describe some such procedure, then show that it does what you claim. For this, we don't actually need a *general definition* of what we mean by a 'mechanical procedure' — it's enough that we know one when we see one, and in particular, that we can all agree that the procedure you've given is indeed of this kind. But what if we want to show that some class of statements is undecidable — that there's *no possible* mechanical procedure for deciding those statements? We couldn't even begin to do this unless we had somehow made precise the general concept of a 'possible mechanical procedure' (or at least, the concept of being decidable by such a procedure). So to make further progress, we need to ask whether there is indeed a plausible way to make this concept precise.

This asymmetry can be seen reflected in the history of the subject. Several interesting decidability results, such as those of Presburger and Skolem, were obtained in the early decades of the 20th century, whereas the first *undecidability* results had to wait for the general concept of decidability to be precisely defined, as was done in the 1930s by Alonzo Church and Alan Turing. It is to this breakthrough that we turn our attention in the next chapter.

Endnotes

1. You might wonder why I've abstained from the symbols − and ÷ in the glyph language. The reason is that the language is cunningly chosen to be suitable for talking about both *Whole* numbers and *Real* numbers. If we're talking about Whole numbers, then subtraction and division aren't always defined: for example, 3−5 and 7÷4 have no Whole number value. (There's also the point that 7÷0 has no Real number value either.) Since part of the point of formal languages is to make it impossible to write nonsense, the simplest solution is just to drop − and ÷ from the language altogether. There's no essential loss in doing this, since anything *sensible* that we might want to say using −, ÷ can always be rephrased using + and ×, as was illustrated in the story.
2. A more formal and precise way of presenting these rules by means of a *context-free grammar* is included in Appendix B.
3. This seems a good point to mention the biography of Tarski by Feferman and Feferman (2004). (I have selectively drawn on this for my portrayal of Tarski in the story.) As well as giving a vivid account of his life and character, this includes elegant and brief summaries of all his main achievements in logic. In particular, the section on Tarski's definition of truth (Interlude III) presents this concept in a somewhat broader context than I have done here.
4. Here are the steps in detail:

    - The values of 3, 4, 5, 7, 6 and 9 are what you expect (Rule 1).

- So the value of (3 + 4) is seven (Rule 2).
- So the value of ((3 + 4) + 5) is twelve (Rule 2 again).
- So the formula ((3 + 4) + 5) = 7 is false (Rule 3).
- The formula 6 = 7 is false (Rule 3).
- So the formula ¬ 6 = 7 is true (Rule 4).
- So the whole formula (((3 + 4) + 5) = 7 ∨ ¬ 6 = 9) is true (Rule 5).

5. The glyph statement is a way of formalizing the Goldbach conjecture: 'Every even number greater than 2 is the sum of two primes'. A closer translation of the formula is as follows: 'For all n, if 2 < n and n is even, then there exist numbers p and q such that n = p+q, p is prime, and q is prime.'
6. Many of the typical ways of using glyph language are things you may well be able to figure out for yourself. But there's one much less obvious trick which it's worth explaining here, because once you're armed with this trick, an enormous number of other things come within easy reach. (The trick is also explained in more detail in the supplement mentioned.)

   The problem that the trick addresses is the following. In glyph language, we can use variables to refer to *individual* Whole numbers, but it's very common in mathematics to want to refer to an entire *sequence* of numbers n_1, n_2, ..., n_r, whose length (the number r) we might not know in advance. To be able to talk about such things in our Whole number glyph language, it would be useful to have a way of 'representing' such a sequence by a *single* Whole number N. (This is a simple example of a *coding* problem: we'll meet several others in Chapter 4 and later.) Naturally, for this to be useful, we'd want things like 'N represents a sequence of length r' and 'The ith element of the sequence represented by N is n' to be expressible in glyph language (using ordinary variables for N, r, i, n).

   Our way of doing this is due to Gödel, and relies on a cute bit of number theory. Let's first see how an arbitrary sequence n_1, n_2, ..., n_r can be faithfully represented by just three numbers m,d,r. Here r is just the length of our sequence, so needs no further explanation.

   The key idea is this. For any j, let's write m mod j for the remainder when m is divided by j (this will be some number less than j). Then we'd be able to recover all the numbers n_1, n_2, ..., n_r from just m and d if, for instance, we knew that

$$m \bmod (1 + d) = n_1, \quad m \bmod (1 + 2d) = n_2, \quad ..., \quad m \bmod (1 + rd) = n_r.$$

   In this case, the numbers m,d,r would together tell us everything about the sequence n_1, n_2, ..., n_r. What's more, the statement 'the ith element of the sequence represented by m,d,r is n' could then be expressed by

$$m \bmod (1 + id) = n$$

   which is something it's quite easy to translate into glyph language.

   The remaining question, then, is whether we *can* always find numbers m and d that encode n_1, n_2, ..., n_r in this way. By a well-known piece of number theory

called the *Chinese remainder theorem*, it turns out that we can, provided we can find a number d such that (a) d is greater than all of n_1, n_2, ..., n_r, and (b) no two of the numbers $1 + d$, $1 + 2d$, ..., $1 + rd$ have any factor in common. As Gödel noted, this is quite easy to arrange: take any k bigger than all of n_1, n_2, ..., n_r and r, and set d to be k ! (that is, $1 \times 2 \times ... \times k$).

This bit of number theory shows that any sequence can be represented by a suitable triple m,d,r. Fortunately, though, one doesn't need to understand the argument in order to construct and use the relevant glyph formulae.

As a final (less essential) step, we might now want to encode the three numbers m,d,r by a single number N. First of all, there's a simple trick for coding a *pair* of numbers: for example, we can represent the pair x,y by the number $(x+y)^2 + y$. (See if you can convince yourself that no two pairs will ever be represented by the same number.) The property 'w represents the pair x,y' is then something we can easily express in glyph language. Applying this idea twice, it's now easy to represent any triple x,y,z by some single number v.

This gives everything we need to be able to talk (in effect) about finite sequences of Whole numbers in our glyph language. And once we've overcome this obstacle — let's call it the *Gödel hurdle* — the way is clear to express all kinds of other things in our language. For instance, you might now like to think about how the property '$z = x^{\,y}$' could be expressed indirectly by a glyph formula. A solution, along with more detail on all of the above, is contained in the supplement *Further tricks in glyph language*.

7. Boole's book, *An Investigation of the Laws of Thought* (1854), languished in obscurity for 80 years until around 1936 it was picked up by the mathematician and engineer Claude Shannon, who realized that Boole's logical operations could actually be physically implemented by electrical switching circuits. The rest, as they say, is history.

8. To see the rationale behind the truth table, it may be helpful to consider a formula such as:

$$x < 5 \;\Rightarrow\; x < 7$$

We clearly want this to come out true, for all values of x. However, it's clear that all of the following three situations are perfectly possible:

- $x < 5$ is false and $x < 7$ is false. (E.g. when $x = 8$)
- $x < 5$ is false by $x < 7$ is true. (E.g. when $x = 6$)
- $x < 5$ is true and $x < 7$ is true. (E.g. when $x = 4$)

So we want $x < 5 \;\Rightarrow\; x < 7$ to come out true in all these cases, which helps to explain why the truth table is the way it is. The only possibility that's ruled out by our formula is that $x < 5$ is true but $x < 7$ is false — which, indeed, doesn't arise for any value of x.

9. There's a question regarding whether equality (=) should be counted as a purely logical notion or not. My view that the boundary between 'logic' and 'mathematics' is a rather fuzzy one anyway, so we can draw the line where we please. My choice in this book has been to classify equality as non-logical.

10. This is easy to do. If we like, we can simply replace

$$\begin{array}{lll} (P \vee Q) & \text{by} & \neg (\neg P \,\&\, \neg Q)\,, \\ (P \Rightarrow Q) & \text{by} & \neg (P \,\&\, \neg Q)\,, \\ (\exists\, x\ P) & \text{by} & \neg (\forall\, x\ \neg P)\,. \end{array}$$

Using Tarski's truth definition — or in the first two cases, the truth tables — we can check that these are indeed valid equivalences. It will also turn out that, in each case, the formula on the left is *provably equivalent* to the one on the right using the logical rules we shall give in Chapter 5.

This way of cutting down to just &, ¬ and ∀ works fine for so-called *classical* logic, but not for *intuitionistic* logic, which we'll come to in Chapter 11.

11. See Montague (1974) for the original papers. A good exposition to Montague's theory appears in Volume 2 of Gamut (1991). The two volumes together offer a wonderful introduction to the whole field of logical approaches to natural language.

I am grateful to Alex Lascarides for helpful discussions of this topic, and for suggesting some of the examples I've mentioned.

12. For example, a problem posed by Ron Graham in the 1980s, the *Boolean Pythagorean triples problem*, was solved by Heule, Kullman and Marek in 2016 with the help of cutting-edge SAT solving techniques, and a serious amount of computing power. This made news headlines as the largest mathematical proof ever constructed: the full propositional proof generated by the computer occupied 200 terabytes, although it could be compressed down to a mere 68 gigabytes.
13. I am indebted to Grant Passmore for introducing me to this example.

In the book, I also promised a discussion of another example drawn from finite projective geometry — an example that enjoys a very pleasing connection with the card game 'Dobble'. I hope to add a discussion of this soon!

14. Fans of Douglas Adams may enjoy the following curiosity. As we've seen, the answers to the problem in two, three and four dimensions are 6, 12 and 24. It has been suggested that there might be some reason why all these numbers are divisible by 6, and this in turn suggests that the answer in five dimensions is, in fact, 42. It's pleasing to reflect that this answer, if correct, could in principle be obtained purely by a computer calculation, albeit one that might take 7,500,000 years to run.
15. An Internet search for 'kissing number' will reveal more information on lower and upper bounds for kissing numbers in various dimensions. Curiously, in 8 dimensions, the kissing number is known exactly (240), and likewise in 24 dimensions (196560). As far as I understand it, this is because some unusually tight packing arrangements are known to be possible in these dimensions.
16. The results on the theory of Real numbers with exponentiation are due to Macintyre and Wilkie. They have given an algorithm, and proved that it is indeed a decision algorithm provided a statement known as *Schanuel's conjecture* is true. This is an important conjecture in the theory of transcendental numbers which is widely believed, but has not yet been proved.